

ЛР18 (3 пары / 6 часов)

«Файл настроек config.json: режим хранения, папки, автосоздание, дефолтные значения»

0) Итог ЛР18 (что должно быть готово)

- ✓ При запуске программа читает `config.json`
 - ✓ Если файла нет — создаёт `config.json` с настройками по умолчанию
 - ✓ Настройки включают:
 - `StorageMode` : `"Json"` или `"Memory"`
 - папки: `DataFolder` , `LogsFolder` , `BackupsFolder` , `ExportsFolder` , `ReportsFolder`
 - ✓ Программа больше не спрашивает "какой режим?" (или спрашивает только если `AskOnStart=true`)
 - ✓ Есть отчёт `docs/LR18_Report.md`
-

1) Теория (важная, понятная)

1.1. Что такое конфиг

Конфиг — файл настроек, который можно менять **без изменения кода**.

Пример: преподаватель хочет:

- всем хранить в JSON
- и чтобы пути были одинаковые

Мы просто меняем config.json.

1.2. Почему JSON для конфига удобно

- человек читаем

- легко сохранять/загружать
- .NET умеет работать с JSON (System.Text.Json)

1.3. Дефолтные значения

Если config.json нет — программа сама создаёт его “правильным”.

Это удобно студентам: “запустил — всё работает”.

2) Практика. Пара 1: создаём модель AppConfig и сервис ConfigService

Шаг 1. Создать файл AppConfig.cs в App

В `TaskTracker.App` создайте папку `Config` :

1. Right click по `TaskTracker.App`
2. Add → New Folder → `Config`

Создайте файл:

1. Right click по папке `Config`
2. Add → Class... → `AppConfig.cs`

Вставьте:

```
namespace TaskTracker.App.Config;

public class AppConfig
{
    public string StorageMode { get; set; } = "Json"; // "Json" или "Memory"
    public bool AskOnStart { get; set; } = false;

    public string DataFolder { get; set; } = "data";
    public string LogsFolder { get; set; } = "logs";
    public string BackupsFolder { get; set; } = "backups";
```

```
public string ExportsFolder { get; set; } = "exports";  
public string ReportsFolder { get; set; } = "reports";  
}
```

Шаг 2. Создать ConfigService.cs

В папке `Config` добавьте класс `ConfigService.cs`:

```
using System.Text.Json;  
  
namespace TaskTracker.App.Config;  
  
public class ConfigService  
{  
    private readonly string _configPath;  
  
    public ConfigService(string configPath)  
    {  
        _configPath = configPath;  
    }  
  
    public AppConfig LoadOrCreateDefault()  
    {  
        if (!File.Exists(_configPath))  
        {  
            var cfg = new AppConfig();  
            Save(cfg);  
            return cfg;  
        }  
  
        try  
        {  
            var json = File.ReadAllText(_configPath);  
            var cfg = JsonSerializer.Deserialize<AppConfig>(j  
son);  
        }  
    }  
}
```

```
        return cfg ?? new AppConfig();
    }
    catch
    {
        // если конфиг сломан – создаём новый дефолтный
        var cfg = new AppConfig();
        Save(cfg);
        return cfg;
    }
}

public void Save(AppConfig cfg)
{
    var json = JsonSerializer.Serialize(cfg, new JsonSerializerOptions
    {
        WriteIndented = true
    });

    File.WriteAllText(_configPath, json);
}
}
```

Шаг 3. Сборка

Build → Build Solution → Build succeeded.

Шаг 4. Коммит №1

Сообщение:

- feat: add AppConfig and ConfigService for config.json

Commit All → Push.

3) Практика. Пара 2: подключаем config.json в Program.cs и берём пути из него

Шаг 5. Открыть Program.cs

TaskTracker.App → Program.cs

Добавьте using:

```
using TaskTracker.App.Config;
```

Шаг 6. Определить путь к config.json

В самом начале Program.cs (после using, перед созданием storage/logger) добавьте:

```
var baseDir = AppContext.BaseDirectory;
var configPath = Path.Combine(baseDir, "config.json");

var configService = new ConfigService(configPath);
var cfg = configService.LoadOrCreateDefault();

Console.WriteLine("Config: " + configPath);
Console.WriteLine($"StorageMode: {cfg.StorageMode}, AskOnStart: {cfg.AskOnStart}");
```

Шаг 7. Пути папок берём из конфига

Замените ваши старые пути:

Было (примерно)

```
var dataFilePath = Path.Combine(AppContext.BaseDirectory, "data", "tasks.json");
var logsFolder = Path.Combine(AppContext.BaseDirectory, "logs");
var backupsFolder = Path.Combine(AppContext.BaseDirectory, "backups");
...
```

Стало

```
var dataFolder = Path.Combine(baseDir, cfg.DataFolder);
var logsFolder = Path.Combine(baseDir, cfg.LogsFolder);
var backupsFolder = Path.Combine(baseDir, cfg.BackupsFolder);
var exportsFolder = Path.Combine(baseDir, cfg.ExportsFolder);
var reportsFolder = Path.Combine(baseDir, cfg.ReportsFolder);

var dataFilePath = Path.Combine(dataFolder, "tasks.json");
```

Шаг 8. Выбор режима хранения из конфига

У вас в ЛР12 был выбор "Json или Memory". Теперь делаем так:

- если `AskOnStart = true` → спрашиваем у пользователя
- иначе → используем `StorageMode` из конфига

Добавьте код:

```
string mode = (cfg.StorageMode ?? "Json").Trim();

if (cfg.AskOnStart)
{
    Console.WriteLine("Выберите хранилище:");
    Console.WriteLine("1 - JSON файл (обычный режим)");
    Console.WriteLine("2 - Memory (данные пропадут после выхода)");
}
```

```
Console.Write("Ваш выбор: ");
var answer = (Console.ReadLine() ?? "").Trim();

mode = answer == "2" ? "Memory" : "Json";

// можно сохранить выбор в конфиг (удобно)
cfg.StorageMode = mode;
configService.Save(cfg);
}
```

Шаг 9. Создаём storage по mode

Замените создание storage на:

```
ITaskStorage storage;

if (string.Equals(mode, "Memory", StringComparison.OrdinalIgnoreCase))
{
    storage = new MemoryTaskStorage();
    Console.WriteLine("Режим: MemoryStorage");
}
else
{
    storage = new JsonTaskStorageAdapter(dataFilePath);
    Console.WriteLine("Режим: JsonStorage");
}
```

Шаг 10. Создаём logger с путём из cfg

```
var logger = new AppLogger(logsFolder);
logger.Info("Application started");
```

Шаг 11. Проверка (F5)

1. Запустить программу
 2. Она должна создать `config.json` (если его не было)
 3. Вывести путь к конфигу
 4. Работать в режиме Json по умолчанию
 5. Остановить, открыть `config.json` и поменять:
 - `"AskOnStart": true`
 6. Запустить снова → теперь спросит режим
-

Шаг 12. Коммит №2

Сообщение:

- `feat: load config.json and use it for paths and storage mode`

Commit All → Push.

4) Практика. Пара 3: команда "Настройки" (показать/изменить) + отчёт

Сделаем пункт меню, чтобы дети могли менять режим без ручного редактирования файла.

Шаг 13. Добавить пункт меню 15

В `ConsoleUi.PrintMenu()` добавьте строку:

```
Console.WriteLine("15) Настройки (показать/сменить режим хранения)");
```

Шаг 14. Реализовать `input == "15"`

В `Program.cs` добавьте блок (лучше через `SafeRunner`):

```

if (input == "15")
{
    SafeRunner.Run("SETTINGS", logger, () =>
    {
        Console.WriteLine("Текущие настройки:");
        Console.WriteLine($"StorageMode: {cfg.StorageMode}");
        Console.WriteLine($"AskOnStart: {cfg.AskOnStart}");
        Console.WriteLine($"DataFolder: {cfg.DataFolder}");
        Console.WriteLine($"LogsFolder: {cfg.LogsFolder}");

        Console.WriteLine();
        Console.WriteLine("Хотите сменить режим хранения?");
        Console.WriteLine("1 - Json");
        Console.WriteLine("2 - Memory");
        Console.Write("Выбор (Enter - не менять): ");
        var ans = (Console.ReadLine() ?? "").Trim();

        if (ans == "1")
        {
            cfg.StorageMode = "Json";
            configService.Save(cfg);
            Console.WriteLine("Сохранено: StorageMode=Json. П
ерезапустите приложение.");
            logger.Info("SETTINGS changed StorageMode=Json");
        }
        else if (ans == "2")
        {
            cfg.StorageMode = "Memory";
            configService.Save(cfg);
            Console.WriteLine("Сохранено: StorageMode=Memory.
Перезапустите приложение.");
            logger.Info("SETTINGS changed StorageMode=Memor
y");
        }
        else

```

```

        {
            Console.WriteLine("Настройки не изменены.");
        }
    });

    continue;
}

```

Мы честно говорим “перезапустите приложение”, потому что storage создаётся при старте.

Шаг 15. Проверка меню настроек

1. Запустить
2. Пункт 15 → выбрать 2 (Memory)
3. Заккрыть программу
4. Запустить снова → убедиться, что режим стал Memory

Шаг 16. Отчёт [docs/LR18_Report.md](#)

Создайте:

```

# Отчёт ЛР18 – config.json

## Что сделано
- Добавлены AppConfig и ConfigService
- Приложение читает config.json при старте
- Если config.json отсутствует или повреждён – создаётся дефолтный
- Пути к папкам (data/logs/backups/exports/reports) берутся из конфига
- Режим хранения берётся из StorageMode (или спрашивается при AskOnStart=true)
- Добавлена команда 15) Настройки для смены StorageMode

```

Как проверить

- 1) Удалить `config.json` и запустить -> файл создаётся
- 2) Поменять `AskOnStart=true` -> приложение спрашивает режим
- 3) Пункт 15 -> сменить `StorageMode` -> перезапустить -> режим изменился

Шаг 17. Коммит №3

Сообщение:

- `feat: add settings command and LR18 report`

Commit All → Push.

5) Что сдавать (чек-лист)

- ✓ `config.json` создаётся автоматически
- ✓ пути берутся из конфига
- ✓ режим хранения берётся из `StorageMode`
- ✓ пункт 15 работает
- ✓ `docs/LR18_Report.md`
- ✓ минимум 3 коммита

6) Оценивание

"3" — `config.json` создаётся и читается, режим работает

"4" — + пути папок из конфига

"5" — + меню настроек (15) и аккуратные сообщения/логи